. ***" STUDENT PERFORMANCE ANALYZER"***

A Project Report submitted
In partial fulfillment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING (DATA SCIENCE)

Submitted by

G. VIKAS © - 2025288655

M. SUJITH VARMA – 2025034833

M. GURUDEV – 2025282602

U.D.S.N. SHANKAR – 2025152481

Under the esteemed guidance of

Dr. Devarapalli Ravi Kishore,Associate Professor, CSE DS

DEPARTMENT OF COMPUTERSCIENCE AND ENGINEERING (DATA SCIENCE)
GITAMSCHOOL OF TECHNOLOGY
GITAM(Deemed to beUniversity)
VISAKHAPATNAM
2025

DEPAR. TMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

GITAMSCHOOLOF TECHNOLOGY

GITAM (Deemedto be University)

DECLARATION

We hereby declare that the project report entitled "STUDENT PERFORMANCE ANALYZER" is an original work done in the Department of Computer Science and Engineering (Data Science), GITAM School of Technology, GITAM (Deemed to be University), submitted in partial fulfillment of the requirements for the award of the degree of B.Tech in Computer Science and Engineering.

The work has not been submitted to any other college or university for the award of any degree or diploma.

Date:

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(DATA SCIENCE)

GITAM SCHOOL OF TECHNOLOGY

GITAM(DeemedtobeUniversity)

CERTIFICATE

This is to certify that the project report entitled "STUDENT PERFORMANCE ANALYZER" is a bonafide record of work carried out by G. Vikas (C), M. Sujith Varma, M. Gurudev, and U.D.S.N. Shankar under my guidance and supervision.

Date :

Project Guide

Dr. Devarapalli Ravi Kishore

Associate Professor

TABLE OF CONTENTS

S.No	*Description*	*PageNo.*
1.	Abstract	5
2.	Introduction	5
3.	Aim of the Project	6
4.	Problem Definition & Objectives	6
5.	Coding	8
6.	Output Screens	18
7.	Future Work	22
8.	Conclusion	23

ABSTRACT

The Student Performance Analyzer Project is a comprehensive data analysis tool designed to simplify the evaluation of student academic performance. Developed using the Python programming language, this application provides a user-friendly, menu-driven interface to perform essential academic analysis tasks.
The system allows users to input student marks, calculate averages, assign grades, compare multiple students, and analyze subject-wise performance. The project utilizes powerful libraries such as Pandas and NumPy for data manipulation and Matplotlib for graphical visualization.
By integrating automated calculations with visual insights, the system helps in understanding student strengths and weaknesses efficiently. This project demonstrates how data analysis techniques can be applied in the education sector to improve decision-making and academic evaluation

INTRODUCTION

In today's education system, analyzing student performance is essential for improving learning outcomes. Traditional methods of evaluating student data manually are time-consuming and prone to human errors.
The Student Performance Analyzer provides a digital solution that automates these tasks. It enables users to input data, perform calculations instantly, and visualize results through graphs and charts.
The project follows a modular programming approach, ensuring clarity and scalability. It also helps students gain practical knowledge in:

- Data Analysis
- Graphical Visualization
- Python Programming
- Problem Solving

AIM OF THE PROJECT

The aim of the Student Performance Analyzer is to design and implement a system that provides an efficient platform to evaluate and analyze student academic performance.
The system helps users to:

- Analyze individual student performance
- Compare multiple students
- Identify top performers
- Understand subject-wise trends

PROBLEM DEFINITION AND OBJECTIVES

PROBLEM DEFINITION

Manual analysis of student performance is difficult and inefficient. Teachers and students often face challenges in understanding performance trends due to lack of proper tools.
There is a need for a system that:

- Automates calculations
- Reduces errors
- Provides clear and visual insights

OBJECTIVES

MAIN OBJECTIVES:

To develop a menu-driven Python application
To analyze student performance efficiently
To provide graphical representation of data

TECHNICAL OBJECTIVES:

- To use Pandas for data handling
- To apply modular programming
- To use Matplotlib for visualization
- To ensure proper input validation

CODING

```
# ==============================
# STUDENT PERFORMANCE ANALYZER
# ==============================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Store student data
students = {}

# ==============================
# Utility Function
# ==============================

def calculate_grade(avg):
    if avg >= 85:
        return "A+"
    elif avg >= 70:
        return "A"
    elif avg >= 55:
        return "B"
    elif avg >= 40:
        return "C"
    else:
        return "Fail"

# ==============================
# 1. Add Student
# ==============================

def add_student():
    name = input("Enter student name: ")
    try:
        marks = list(map(int, input("Enter marks of 5 subjects: ").split()))
    except:
        print("Invalid input")
        return

    if len(marks) != 5:
        print("Enter exactly 5 subjects")
        return

    students[name] = marks
    print("Student added successfully!")
```

```
# ==============================
# 2. Student Analysis
# ==============================

def student_analysis():
name = input("Enter student name: ")

if name not in students:
print("Student not found")
return

marks = students[name]
avg = sum(marks) / len(marks)
grade = calculate_grade(avg)

print("\n----- Student Report -----")
print("Marks:", marks)
print("Average:", round(avg, 2))
print("Grade:", grade)

# ==============================
# 3. Compare Students
# ==============================

def compare_students():
if len(students) == 0:
print("No students available")
return

averages = {}

for name, marks in students.items():
avg = sum(marks) / len(marks)
averages[name] = avg

best_student = max(averages, key=averages.get)

print("\nStudent Averages:")
for k, v in averages.items():
print(k, ":", round(v, 2))

print("\nTop Performer:", best_student)
```

```
# Graph
plt.figure()
plt.bar(averages.keys(), averages.values())
plt.title("Student Comparison")
plt.xlabel("Students")
plt.ylabel("Average Marks")
plt.grid()
plt.show()

# ==============================
# 4. Class Performance
# ==============================

def class_performance():
    if len(students) == 0:
        print("No data available")
        return

    averages = [sum(m)/len(m) for m in students.values()]

    print("Class Average:", round(sum(averages)/len(averages), 2))

    plt.figure()
    plt.plot(averages, marker='o')
    plt.title("Class Performance Trend")
    plt.xlabel("Students Index")
    plt.ylabel("Average Marks")
    plt.grid()
    plt.show()

# ==============================
# 5. Subject Analysis
# ==============================

def subject_analysis():
    if len(students) == 0:
        print("No data available")
        return

    data = pd.DataFrame(students).T
    subject_avg = data.mean()

    print("\nSubject-wise Average:")
    print(subject_avg)

    plt.figure()
    plt.bar(range(1, 6), subject_avg)
    plt.title("Subject Performance")
    plt.xlabel("Subjects")
    plt.ylabel("Average Marks")
    plt.grid()
    plt.show()
```

```
# ==============================
# Main Menu
# ==============================

while True:
print("\n===== STUDENT PERFORMANCE ANALYZER =====")
print("1. Add Student")
print("2. Student Analysis")
print("3. Compare Students")
print("4. Class Performance")
print("5. Subject Analysis")
print("6. Exit")

choice = input("Enter choice: ")

if choice == "1":
add_student()
elif choice == "2":
student_analysis()
elif choice == "3":
compare_students()
elif choice == "4":
class_performance()
elif choice == "5":
subject_analysis()
elif choice == "6":
print("Exiting...")
break
else:
print("Invalid choice")
```

OUTPUT SCREEN

1. Main Menu Display

When the program starts, the system displays a menu-driven interface with multiple options such as:

Add Student Data

Display Student Details

Calculate Average Marks

Assign Grades

Compare Students

Exit

This allows the user to interact with the system easily.

2. Student Data Entry

The user is prompted to enter the following details:

Student Name

Roll Number

Marks in different subjects

The system validates the input and stores the data for further processing

3. Display of Student Details

After entering the data, the system displays:

Student Name

Roll Number

Subject-wise Marks

This helps in verifying whether the data entered is correct.

4. Average Marks Calculation

The system calculates the average marks of the student using the formula:

Average = (Sum of all subject marks) / Number of subjects

The calculated average is displayed clearly.

5. Grade Assignment

Based on the average marks, the system assigns grades:

A Grade → Above 90

B Grade → 75 – 89

C Grade → 50 – 74

D Grade → Below 50

This helps in evaluating student performance effectively.

6. Comparison of Students

The system allows comparison between multiple students based on:

Average marks

Overall performance

The top-performing student is identified and displayed.

7. Graphical Representation

The system uses graphs to represent data visually:

Bar charts for subject-wise marks

Comparison graphs for multiple students

Performance trends

These visualizations make it easier to understand the results.

8. Program Exit

After completing all operations, the user can safely exit the program using the exit option from the menu.

FUTURE WORK

The current system can be enhanced further by implementing:

Graphical User Interface (GUI) using Tkinter

Database integration for storing student records

Machine learning models for predicting performance

Web-based dashboard for remote access

CONCLUSION

The Student Performance Analyzer project successfully demonstrates how Python can be used to simplify academic data analysis. The system automates calculations, reduces errors, and provides meaningful insights through graphical visualization.

It serves as a practical example of applying programming and data analysis techniques in the field of education. The project enhances understanding of both technical and analytical concepts, making it a valuable learning experience.